# SystemVerilog 父类与子类

**变量隐藏 • 约束继承 • 方法多态 • 访问控制**

Parent & Child Classes — Variable Hiding, Constraint Inheritance, Polymorphism & Access Control

## 1. 同名变量与约束 Same-name variables & constraints

当子类重新定义一个父类已存在的变量时，SystemVerilog 把它们当作**两个独立的变量**（variable hiding，变量隐藏）。父类和子类各自保留自己的版本以及各自的约束，互不影响。

| 访问位置 | 如何访问 |
| --- | --- |
| 子类中的 `addr` | 直接写 `addr` |
| 父类中的 `addr` | 用 `super.addr` |

> **核心结论：**句柄类型决定编译期能访问哪个成员；但 `randomize()` 是虚方法，按对象的实际类型分发。两个同名变量是真正独立的，各自被各自的约束约束。

### 1.1 变量隐藏 + 不同名约束

父类定义 `addr` 并约束为 0；子类隐藏它，重新定义 `addr` 并给出自己的约束。

```
// 父类 — lvc_apb_master_base_sequence.sv
class lvc_apb_master_base_sequence extends uvm_sequence #(lvc_apb_transfer);
  rand bit [31:0] addr;
  constraint addr_zero { addr == 32'd0; }
endclass

// 子类 — lvc_apb_master_burst_write_sequence.sv
class lvc_apb_master_burst_write_sequence extends lvc_apb_master_base_sequence;
  rand bit [31:0] addr;          // 隐藏父类的 addr
  rand bit [31:0] data[];
  lvc_apb_trans_status trans_status;

  constraint cstr {
    soft data.size() inside {4, 8, 16, 32};
    foreach (data[i]) soft data[i] == addr + (i << 2);
  }
endclass
```

对子类对象调用 `randomize()` 之后：父类的 `addr` 因 `addr_zero` 保持 0；子类的 `addr` 按 `cstr` 被随机化。二者独立。

### 1.2 完整可运行示例

父类和子类各自定义 `a`，各自的约束（同名约束 `ct_a`，但作用在不同的隐藏变量上）。

```
module junk();
  class base;
    rand bit [15:0] a, b;
    function new(); endfunction
    constraint ct_a { a inside {[0:9]}; }
  endclass

  class extended extends base;
    rand bit [15:0] a, b;                    // 隐藏父类 a
    function new(); super.new(); endfunction
    constraint ct_a { a inside {[10:19]}; }
    function int get_parent_value(); return super.a; endfunction
  endclass

  initial begin
    extended extended_obj;
    base     base_obj;
    extended_obj = new();
    base_obj     = new();
    if (!extended_obj.randomize()) $display("Randomization Failed for extended_obj");
    if (!base_obj.randomize())     $display("Randomization Failed for base_obj");
    $display("Extended class a: %0d", extended_obj.a);
    $display("Base class a: %0d",     base_obj.a);
    $display("Parent's a from extended: %0d", extended_obj.get_parent_value());
  end
endmodule
```

```
// 输出
Extended class a: 12
Base class a: 0
Parent's a accessed from extended: 55225 // super.a 未被随机化，是构造后的残值
```

子类对象随机化的是**子类的** `a`（10–19）。父类那份 `a` 从没被这次 randomize 触及，`super.a` 读到的是未随机化的值。

### 1.3 多态：父类句柄指向子类对象

把子类对象赋给父类句柄后再 `randomize()`，本质仍是对**子类对象**随机化——所有 rand 成员和所有生效约束都参与；但**通过父类句柄只能访问父类那份变量**。

```
initial begin
  extended extended_obj;
  base     base_obj;
  extended_obj = new();
  base_obj = extended_obj;            // 上溯造型：父类句柄指向子类对象
  if (!base_obj.randomize()) $display("Randomization Failed");
  $display("Extended class a: %0d", extended_obj.a);   // 子类 a
  $display("Base class a: %0d",     base_obj.a);       // 通过父类句柄 -> 父类 a
end
```

```
// 输出
Extended class a: 12
Base class a: 55225
Parent's a accessed from extended: 55225
```

`base_obj.a` 打印的是**父类那份** `a`（句柄类型决定成员），而 `base_obj.randomize()` 却随机化了**整个子类对象**（虚分发）。两条规则不矛盾——一个管成员访问，一个管随机化分发。

### 1.4 只有父类声明变量

子类不再隐藏 `a`，只添加约束。此时父类和子类的约束作用于**同一个** `a`。

### ① 不同名约束（`ct_a` vs `ct_b`）

```
class base;
  rand bit [15:0] a, b;
  constraint ct_a { a inside {[2:9]}; }     // 或 {[0:9]}
endclass
class extended extends base;
  // 不再定义 a
  constraint ct_b { a inside {[10:19]}; }
endclass
```

**父类对象**随机化：只有 `ct_a` 生效 → `a` ∈ 2–9，子类不参与。

**子类对象**随机化：`ct_a` 与 `ct_b` **同时作用于同一个** `a` → `[0:9]` 与 `[10:19]` 无交集 → **约束冲突，randomize 失败**。

```
// 输出
// 父类对象：Base class a: 2
// 子类对象：Randomization Failed（ct_a 与 ct_b 冲突，无解）
```

### ② 同名约束（都叫 `ct_a`）

```
class base;
  rand bit [15:0] a, b;
  constraint ct_a { a inside {[2:9]}; }
endclass
class extended extends base;
  constraint ct_a { a inside {[10:19]}; }   // 同名 -> override 父类约束
endclass
```

**父类对象**随机化：正常用父类 `ct_a` → `a` ∈ 2–9。

**子类对象**随机化：子类 `ct_a` **override（覆盖）**父类同名约束 → 只剩 10–19，父类那条已**失效**，因此**不会冲突**。

```
// 输出
Extended class a: 12
Base class a: 2
Parent's a accessed from extended: 12
```

> **一句话规则：**约束**同名 → override**（父类那条失效，永不冲突）；**不同名 → 累加**（逻辑 AND，可能冲突导致失败）。这是本节最容易记错的点。

| 情形 | 父类约束 | 结果 |
| --- | --- | --- |
| 子类约束 **同名** | 被覆盖，失效 | 只有子类约束生效 |
| 子类约束 **不同名** | 仍生效 | 两者 AND；矛盾则 randomize 失败 |
| 同名 + 矛盾 | 失效 | 无冲突，只用子类的 |
| 不同名 + 矛盾 | 生效 | 无解 → randomize 返回 0 |

## 2. 同名方法与多态 Same-name methods & polymorphism

与变量不同，子类中同名方法**不是隐藏**而是**覆写（override）**。是否动态绑定取决于方法有没有 `virtual` 关键字。用 `super.method()` 显式调用父类版本。

### 2.1 非 virtual 方法 — 静态绑定

调用哪个方法取决于**句柄的静态类型**，与对象实际类型无关。

```
class base;
  function void print_info(); $display("Base class print_info called"); endfunction
endclass
class extended extends base;
  function void print_info(); $display("Extended class print_info called"); endfunction
endclass

initial begin
  extended ext_obj = new();
  base     base_obj = new();
  base     base_handle;
  base_handle = ext_obj;      // 多态赋值
  ext_obj.print_info();       // 子类方法
  base_obj.print_info();      // 父类方法
  base_handle.print_info();   // 静态绑定 -> 父类方法！
end
```

```
// 输出
Extended class print_info called
Base class print_info called
Base class print_info called // 非 virtual：句柄类型决定，调用父类
```

## 2.2 virtual 方法 — 动态绑定

调用哪个方法取决于**对象的实际类型**。即使通过父类句柄，也执行子类覆写版本——这是多态的核心。

```
class base;
  virtual function void print_info(); $display("Base class print_info called"); endfunction
endclass
class extended extends base;
  virtual function void print_info();   // virtual 可省略，继承自父类
    $display("Extended class print_info called");
  endfunction
endclass

initial begin
  extended ext_obj = new();
  base     base_obj = new();
  base     base_handle;
  base_handle = ext_obj;
  ext_obj.print_info();       // 子类方法
  base_obj.print_info();      // 父类方法
  base_handle.print_info();   // 动态绑定 -> 子类方法！
end
```

```
// 输出
Extended class print_info called
Base class print_info called
Extended class print_info called // virtual：动态绑定，调用子类
```

## 2.3 用 `super` 保留父类逻辑

子类覆写时可先调 `super.method()`，在扩展行为的同时保留父类逻辑。

```
class extended extends base;
  virtual function void print_info();
    super.print_info();                          // 先调父类
    $display("Extended class print_info called");    // 再加子类逻辑
  endfunction
endclass
```

```
// 输出
Base class print_info called
Extended class print_info called
```

### 2.4 子类未覆写

子类继承父类方法，直接调用即可；`super.method()` 效果相同。

| 调用场景 | 非 virtual | virtual |
| --- | --- | --- |
| 子类句柄调用 | 子类方法 | 子类方法 |
| 父类句柄（指向子类对象） | 父类方法（静态绑定） | 子类方法（动态绑定） |
| `super.method()` | 父类方法 | 父类方法 |

## 3. 变量的访问控制 Variable accessibility

父类与子类之间的变量可见性由 `local` / `protected` / `public`（默认）决定。

| 修饰符 | 父类内部 | 子类内部 | 类外部（句柄） |
| --- | --- | --- | --- |
| 无（public） | ✓ | ✓ | ✓ |
| protected | ✓ | ✓ | ✗ |
| local | ✓ | ✗ | ✗ |

### 3.1 三种修饰符

```
class base;
  bit [15:0]           pub_var;   // public，默认：内外皆可访问
  protected bit [15:0] prot_var;  // 父类+子类内部可访问，外部不可
  local     bit [15:0] local_var; // 仅父类内部；子类与外部均不可，需经 getter
endclass
```

> `local` 变量子类也读不到——只能通过父类提供的 public/protected 方法（getter）间接访问。

### 3.2 子类访问父类变量：`super`

当子类定义了同名变量（隐藏）时，用 `super.var` 访问父类那份；无同名时直接访问即可。

```
class base;
  bit [15:0] a;
  function new(); a = 16'h1111; endfunction
endclass
class extended extends base;
  bit [15:0] a;                             // 隐藏父类 a
  function new(); super.new(); a = 16'h2222; endfunction
  function void show();
    $display("Subclass a: %0h", a);         // 子类 a
    $display("Parent a (super): %0h", super.a); // 父类 a
  endfunction
endclass
```

```
// 输出
Subclass a: 2222
Parent a (super): 1111
```

### 3.3 父类句柄访问子类变量：需 `$cast` 下溯

父类句柄**不能**直接访问子类新增变量，即使它指向子类对象。必须先 `$cast` 下溯为子类类型。

```
class base;
  bit [15:0] a;
  function new(); a = 16'h1111; endfunction
endclass
class extended extends base;
  bit [15:0] b;                              // 子类新增
  function new(); super.new(); b = 16'h2222; endfunction
endclass

initial begin
  extended ext_obj = new();
  base     base_handle;
  extended cast_obj;
  base_handle = ext_obj;
  $display("base_handle.a = %0h", base_handle.a);   // 合法：父类变量
  // $display(base_handle.b);                        // 非法：父类句柄看不到 b
  if ($cast(cast_obj, base_handle))                  // 下溯造型（运行期检查）
    $display("After $cast, cast_obj.b = %0h", cast_obj.b);  // 合法
end
```

```
// 输出
base_handle.a = 1111
After $cast, cast_obj.b = 2222
```

> **$cast 下溯的前提：**对象**实际就是**子类类型（本例由子类 `new()` 创建后上溯而来）。若对象本身是纯父类，`$cast` 会失败——它**检查**类型，不**转换**对象。

## 4. 方法的访问控制 Method accessibility

方法可见性遵循与变量相同的 `local` / `protected` / `public` 规则。`super` 显式调父类版本，`$cast` 下溯后可调用子类独有方法。

### 4.1 子类调用父类方法

```
class base;
  virtual function void greet(); $display("Hello from Base"); endfunction
  function void base_only(); $display("This method is only in Base"); endfunction
endclass
class extended extends base;
  virtual function void greet();
    super.greet();                       // 显式调父类
    $display("Hello from Extended");
  endfunction
  function void call_base_only(); base_only(); endfunction  // 直接调继承方法
endclass
```

```
// 输出
Hello from Base
Hello from Extended
This method is only in Base
```

### 4.2 父类句柄调用子类方法

父类句柄**不能**直接调用子类新增（父类不存在）的方法，需 `$cast` 下溯。但对父类中声明为 `virtual` 的方法，父类句柄调用时会**动态绑定**到子类覆写版本。

```
initial begin
  extended ext_obj = new();
  base     base_handle;
  extended cast_obj;
  base_handle = ext_obj;
  base_handle.greet();                    // virtual：动态绑定 -> 子类 greet
  // base_handle.extended_only();          // 非法：父类句柄看不到子类新方法
  if ($cast(cast_obj, base_handle))
    cast_obj.extended_only();             // 下溯后合法
end
```

```
// 输出
Hello from Extended
Extended-only method called
```

### 4.3 local / protected 方法

`local` 方法仅定义它的类内部可调；子类与外部均不可，只能经 public 方法间接调用。`protected` 方法父类与子类内部可调，外部不可。

| 访问场景 | public | protected | local |
|---|---|---|---|
| 父类内部 | ✓ | ✓ | ✓ |
| 子类内部（继承/super） | ✓ | ✓ | ✗ |
| 类外部（句柄） | ✓ | ✗ | ✗ |
| 父类句柄调子类**新**方法 | ✗（需 $cast） | ✗ | ✗ |
| 父类句柄调 virtual 覆写方法 | ✓（动态绑定） | ✓（动态绑定） | ✗ |

## 速查总结 Cheat sheet

| 主题 | 关键规则 |
|---|---|
| 同名**变量** | 隐藏（hiding）——父子两份独立，子类用 `super.x` 访问父类那份 |
| 同名**约束** | override——子类同名约束覆盖父类，永不冲突 |
| 不同名约束 | 累加（AND）——矛盾则 randomize 失败 |
| 同名**方法** | 覆写（override）——virtual 动态绑定，非 virtual 静态绑定 |
| `randomize()` | 虚分发——按对象实际类型，随机化整个子类对象及全部生效约束 |
| 成员访问 | 按句柄静态类型——父类句柄只能看父类成员 |
| 下溯 `$cast` | 运行期类型检查——对象须实际为子类，否则失败 |